

مستندات یلاگین یونیتا (Unita) 1.0.0 یونیتی

توسعه دهنده: سجاد احمدی نیت (صنایع خلاق نسیم مهر، استودیو رز یخی)

1. معرفی پروژه

این پروژه اولین پروژه زیرساخت نرم‌افزاری و میان‌افزار (Middleware / Bridge) اختصاصی ایتا برای یونیتی است که با هدف تبدیل بازی‌ها و نرم‌افزارهای توسعه‌یافته در موتور بازی‌سازی **Unity** به برنامه‌های تعاملی تحت وب (Mini Apps / WebApps) در بستر پیام‌رسان ایتا طراحی و پیاده‌سازی شده است.

در این معماری، خروجی WebGL یونیتی از طریق یک قالب وب استاندارد (WebGL Template) بارگذاری شده و با استفاده از یک لایه ارتباطی مبتنی بر جاوااسکریپت (jslib) و سی‌شارپ (EitaaManager.cs)، تعامل دوطرفه و کاملی با **SDK رسمی پیام‌رسان ایتا** (eitaa-web-app.js) برقرار می‌کند.

2. معماری پروژه

ساختار کلی پروژه به شکل زیر است:

```
Unity Project/
├── Assets/
│   ├── Plugins/
│   │   ├── WebGL/
│   │   │   ├── UnitaBridge.jslib
│   │   │   └──
│   └── Scripts/
│       ├── UnitaManager.cs
│       └──
├── WebGLTemplates/
│   ├── EitaaTemplate/
│   │   ├── index.html
│   │   ├── unita-background.png
│   │   ├── thumbnail.png
│   │   └── unita-loading-font.woff2
```

و خروج یونیتی به WebGL هم به شکل زیر است:

└── WebGL/ خروجی
 ├── index.html
 ├── Build/
 ├── TemplateData/
 └── StreamingAssets/

3. پیش نیازهای فنی

برای اجرای برنامه به موارد زیر نیاز دارید:

- Unity با پشتیبانی WebGL (ماژول WebGL باید نصب باشد)
 - پروژه‌ای که در WebGL قابل Build باشد
 - دامنه دارای گواهی SSL
 - آدرس HTTPS بدون ریدایرکت
 - هاست مناسب برای فایل‌های WebGL
 - حساب توسعه‌دهنده در پنل ایتایار (یا استفاده از برنامه آن)
 - نسخه اندروید یا دسکتاپ ویندوز ایتا برای تست
- برنامه ایتا یک برنامه مستقل نصب‌شونده نیست؛ بلکه یک WebApp است که در WebView داخلی ایتا اجرا می‌شود.

4. فایل قالب (index.html)

فایل index.html نقطه ورود خروجی WebGL است و وظایف زیر را انجام می‌دهد:

1. ایجاد صفحه HTML برای اجرای Unity

2. ایجاد عنصر canvas

3. بارگذاری فایل Loader یونیتی

4. بارگذاری SDK رسمی ایتا

5. نمایش نوار پیشرفت

6. تنظیم ابعاد Canvas

7. نمایش خطاهای زمان بارگذاری

8. ایجاد Unity Instance

5. مسیر فایل قالب

فایل را در مسیر زیر قرار دهید:

Assets/WebGLTemplates/EitaaTemplate/index.html

ساختار دقیق:

Assets

└── WebGLTemplates

└── EitaaTemplate

└── index.html

نام‌ها باید دقیقاً انگلیسی باشند و بین آن‌ها فاصله وجود نداشته باشد.

5. اتصال SDK ایتا

کد زیر باید داخل <head> قرار بگیرد:

```
<script src="https://developer.eitaa.com/eitaa-web-app.js"></script>
```

این فایل باید مستقیماً از سرور رسمی ایتا بارگذاری شود.

نکته امنیتی

فایل SDK را:

- دانلود نکنید؛
- داخل پروژه ذخیره نکنید؛
- تغییر ندهید؛
- نسخه محلی آن را در هاست خود قرار ندهید.

دلیل این کار، دریافت خودکار به‌روزرسانی‌ها و اصلاحات امنیتی SDK است.

6. متغیرهای اختصاصی Unity

در قالب HTML یونیتی از متغیرهای زیر استفاده می‌شود:

{{{ PRODUCT_NAME }}} : نام محصول

{{{ PRODUCT_VERSION }}} : نسخه محصول

{{{ COMPANY_NAME }}} : نام شرکت یا سازنده

{{{ LOADER_FILENAME }}} : نام فایل بارگذار

{{{ DATA_FILENAME }}} : نام فایل داده

{{{ FRAMEWORK_FILENAME }}} : نام فایل چهارچوب

{{{ CODE_FILENAME }}} : نام فایل اسکریپت

این متغیرها هنگام Build توسط Unity جایگزین می‌شوند.

برای مثال:

نمایش کد

```
const loaderUrl = buildUrl + "/{{{ LOADER_FILENAME }}}";
```

پس از Build ممکن است به چیزی شبیه این تبدیل شود:

```
const loaderUrl = "Build/MyGame.loader.js";
```

این متغیرها را دستی با نام فایل واقعی جایگزین نکنید؛ زیرا هر بار نام فایل‌ها ممکن است تغییر کند.

7. بخش تنظیم Canvas در HTML

کد HTML زیر باعث می‌شود Canvas تقریباً تمام فضای WebView را بگیرد:

```
body {  
    width: 100%;  
    height: 100%;
```

```
margin: 0;
padding: 0;
overflow: hidden;
}
#unity-container {
    position: fixed;
    inset: 0;
    width: 100%;
    height: 100%;
}
#unity-canvas {
    display: block;
    width: 100%;
    height: 100%;
}
```

در اینجا:

- width: 100% یعنی استفاده از تمام عرض WebView؛
- height: 100% یعنی استفاده از تمام ارتفاع WebView؛
- overflow: hidden مانع ایجاد اسکرول ناخواسته می‌شود؛
- Canvas position: fixed را نسبت به کل پنجره ثابت نگه می‌دارد.

8. فایل EitaaBridge.jslib

8.1 وظیفه فایل

فایل jslib. پل ارتباطی میان C# یونیتی و JavaScript است.

Unity مستقیماً نمی‌تواند هر تابع JavaScript را از C# فراخوانی کند. برای این کار:

1. تابع JavaScript را در فایل jslib. تعریف می‌کنیم؛

2. آن را در LibraryManager.library ثبت می‌کنیم؛

3. در C# با [DllImport("__Internal")] همان تابع را اعلام می‌کنیم؛

4. در زمان اجرای WebGL، Unity آن تابع را فراخوانی می‌کند.

8.2 مسیر فایل

فایل را در این مسیر قرار دهید:

Assets/Plugins/WebGL/EitaaBridge.jslib

نام فایل EitaaBridge.jslib پسوند فایل باید واقعاً jslib. باشد.

نه EitaaBridge.jslib.txt

8.3 ساختار عمومی فایل

ساختار اصلی فایل به صورت زیر است:

Script

```
}  
});  
`, {  
    FunctionName: function () {  
        //statement  
    }  
});
```

نام FunctionName باید با نام تابع تعریف‌شده در C# یکسان باشد.

8.4 تبدیل رشته بین C# و JavaScript

در JavaScript برای تبدیل Pointer به رشته باید از این تابع استفاده شود:

UTF8ToString(pointer)

مثال:

```
EitaaSetHeaderColor: function (colorPtr) {  
    var color = UTF8ToString(colorPtr);  
};
```

در سمت C#:

```
[DllImport("__Internal")]  
private static extern void EitaaSetHeaderColor(string colorHex);
```

8.5 ارسال نتیجه از JavaScript به Unity

برای ارسال نتیجه به یک GameObject در Unity از SendMessage استفاده می‌شود:

```
SendMessage(  
    objectName,  
    callbackName,  
    result  
);
```

پارامتر	توضیح
objectName	نام GameObject در Unity
callbackName	نام متد عمومی C#
result	داده‌ای که باید به C# ارسال شود

مثال:

```
SendMessage("EitaaManager", "OnReceiveInitData", initData);
```

در Unity باید متدی با این مشخصات وجود داشته باشد:

```
public void OnReceiveInitData(string rawData)
```

```
{  
    Debug.Log(rawData);  
}
```

9. کلاس ها

کلاس EitaaManager.cs

کلاس EitaaManager وظایف زیر را بر عهده دارد:

- اعلام آماده بودن برنامه
- گسترش برنامه
- بستن برنامه
- دریافت initData
- دریافت نسخه SDK
- تغییر رنگ Header
- تغییر رنگ Background
- نمایش Alert
- تنظیم Main Button
- اجرای Haptic Feedback
- فراهم کردن API واحد برای سایر اسکریپت های پروژه

10. افزودن اسکریپت به صحنه

در اولین Scene پروژه:

1. در پنجره Hierarchy راست کلیک کنید؛

2. گزینه Create Empty را انتخاب کنید.

نام آن را بگذارید: EitaaManager

4. اسکریپت EitaaManager.cs را روی آن Drag کنید؛

5. مطمئن شوید این GameObject هنگام ورود به Scene حذف نمی‌شود.

کلاس از Singleton استفاده می‌کند و از طریق زیر قابل دسترسی است:

```
EitaaManager.Instance
```

چرا از Singleton استفاده شده است؟

با Singleton نیازی نیست در همه Scene ها نمونه جدیدی از Manager بسازید. مثلاً:

```
EitaaManager.Instance.Expand();
```

همچنین:

```
DontDestroyOnLoad(gameObject);
```

باعث می‌شود Manager هنگام تغییر Scene باقی بماند.

11. حالت WebGL و Editor

توابع JavaScript در Unity Editor وجود ندارند. بنابراین استفاده از این شرط

ضروری است:

```
#if UNITY_WEBGL && !UNITY_EDITOR
```

```
EitaaReady();
```

```
#else
```

```
Debug.Log("[Eitaa] Ready called in Editor");
```

```
#endif
```

نتیجه:

- در WebGL واقعی، تابع JavaScript اجرا می‌شود؛
- در Unity Editor، برنامه خطا نمی‌دهد و فقط پیام Debug نمایش می‌دهد.

12. مستندات متدهای EitaaManager

12.1 متد Ready

```
public void Ready()
```

کاربرد: به محیط ایتا اعلام می‌کند که برنامه بارگذاری شده و آماده استفاده است.

استفاده:

```
EitaaManager.Instance.Ready();
```

زمان مناسب اجرا:

معمولاً در Start یا پس از آماده‌شدن رابط اصلی:

```
private void Start()
{
    EitaaManager.Instance.Ready();
}
```

12.2 متد Expand

```
public void Expand()
```

کاربرد: برنامه را تا حداکثر ارتفاع قابل استفاده WebView گسترش می‌دهد.

استفاده:

```
EitaaManager.Instance.Expand();
```

کاربرد پیشنهادی: پس از ورود کاربر به صفحه اصلی:

```
private void Start()
{
    EitaaManager.Instance.Ready();
    EitaaManager.Instance.Expand();
}
```

12.3 متد CloseWebApp

```
public void CloseWebApp()
```

کاربرد: برنامه را می‌بندد.

استفاده:

```
EitaaManager.Instance.CloseWebApp();
```

اتصال به دکمه Unity

1. یک Button در Canvas بسازید؛

2. در بخش On Click()، شیء EitaaManager را قرار دهید؛

3. تابع زیر را انتخاب کنید:

```
EitaaManager > CloseWebApp()
```

12.4 متد RequestInitData

```
public void RequestInitData()
```

کاربرد: رشته خام احراز هویت کاربر را از SDK ایتا دریافت می‌کند.

استفاده:

```
EitaaManager.Instance.RequestInitData();
```

نتیجه در متد زیر دریافت می‌شود:

```
public void OnReceiveInitData(string rawData)
```

اهمیت امنیتی:

initData باید به سرور ارسال و در سمت سرور اعتبارسنجی شود.

نباید فقط به اطلاعات دریافت‌شده از سمت کلاینت اعتماد کنید.

12.5 متد OnReceiveInitData

```
public void OnReceiveInitData(string rawData)
```

کاربرد: Callback دریافت پاسخ از initData JavaScript است.

نمونه استفاده:

```
public void OnReceiveInitData(string rawData)
{
    if (string.IsNullOrEmpty(rawData))
    {
        Debug.LogWarning("initData نشد");
        Log("initData: " + rawData);
    }
}
```

هشدار: این متد را مستقیماً از UI صدا نزنید. این متد باید توسط JavaScript فراخوانی شود.

12.6 متد RequestVersion

```
public void RequestVersion()
```

کاربرد: نسخه SDK ایتا را دریافت می‌کند.

استفاده:

```
EitaaManager.Instance.RequestVersion();
```

پاسخ در این متد قرار می‌گیرد:

```
public void OnReceiveVersion(string version)
```

12.7 متد SetHeaderColor

```
public void SetHeaderColor(string colorHex)
```

کاربرد: رنگ Header برنامه را تغییر می‌دهد.

فرمت معتبر: "#202124"

نمونه: EitaaManager.Instance.SetHeaderColor("#1E1E24");

نکته:

رنگ باید معمولاً به صورت شش رقمی باشد: #RRGGBB

نمونه صحیح:

#FFFFFF

#000000

#2196F3

12.8 متد SetBackgroundColor

public void SetBackgroundColor(string colorHex)

کاربرد: رنگ پس‌زمینه محیط برنامه ایتا را تنظیم می‌کند.

نمونه:

EitaaManager.Instance.SetBackgroundColor("#202124");

برای هماهنگ‌سازی با رنگ محیط Unity:

EitaaManager.Instance.SetBackgroundColor("#101018");

EitaaManager.Instance.SetHeaderColor("#101018");

12.9 متد ShowAlert

public void ShowAlert(string message)

کاربرد: نمایش پیام هشدار یا اطلاع‌رسانی در محیط ایتا.

نمونه:

EitaaManager.Instance.ShowAlert("عملیات با موفقیت انجام شد.");

در Editor، به جای پنجره Native، پیام در Console ثبت می‌شود.

12.10 متد ConfigureMainButton

```
public void ConfigureMainButton(  
    string text,  
    string colorHex,  
    string textColorHex,  
    bool isVisible,  
    bool isActive  
)
```

کاربرد: تنظیم کردن دکمه اصلی

پارامترها:

پارامتر	توضیح
text	متن دکمه
colorHex	رنگ پس‌زمینه دکمه
textColorHex	رنگ متن
isVisible	نمایش یا عدم نمایش
isActive	فعال یا غیرفعال بودن

نمونه:

```
EitaaManager.Instance.ConfigureMainButton(  
    "شروع بازی",  
    "#2196F3",  
    "#FFFFFF",  
    true,  
    true  
);
```

برای مخفی کردن:

```
EitaaManager.Instance.ConfigureMainButton(  
    "",  
    "#2196F3",  
    "#FFFFFF",  
    false,  
    false  
);
```

12.11 متد TriggerHaptic

```
public void TriggerHaptic(string style = "light")
```

کاربرد: فعال کردن بازخورد لمسی دستگاه.

سبک‌های معمول:

light

medium

heavy

rigid

soft

نمونه:

```
EitaaManager.Instance.TriggerHaptic("light");
```

یا:

```
EitaaManager.Instance.TriggerHaptic("medium");
```

برای کلیک دکمه:

```
public void OnButtonClicked()
{
    EitaaManager.Instance.TriggerHaptic("light");
}
```

13. رویدادهای #C و دریافت داده

کلاس می‌تواند رویدادهایی مانند این موارد داشته باشد:

```
public static event Action<string> OnInitDataReceived;
public static event Action<string> OnVersionReceived;
```

برای دریافت رویداد:

```
private void OnEnable()
{
    EitaaManager.OnInitDataReceived += HandleInitData;
}
```

```
private void OnDisable()
{
    EitaaManager.OnInitDataReceived -= HandleInitData;
}
```

```
private void HandleInitData(string data)
{
    Debug.Log("اطلاعات کاربر دریافت شد: " + data);
}
```


}

چرا باید در OnDisable لغو اشتراک (عضویت) کنیم؟

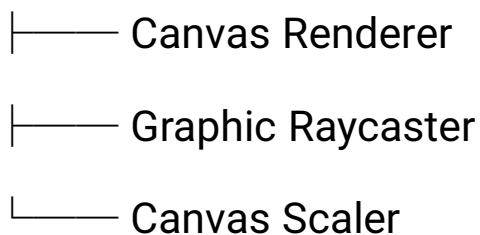
برای جلوگیری از:

- اجرای چندباره Callback؛
- باقی ماندن Reference ای اضافی؛
- خطا هنگام تغییر Scene؛
- افزایش مصرف حافظه.

14. تنظیم Canvas در Unity

برای Canvas اصلی:

Canvas



تنظیمات Canvas

Render Mode

برای رابط کاربری دوبعدی:

Screen Space - Overlay

این حالت پیشنهاد اصلی برای منوها، دکمه‌ها و HUD است.

Canvas Scaler

UI Scale Mode: Scale With Screen Size

Reference Resolution: 1080 x 1920

Screen Match Mode: Match Width Or Height

Match: 0.5

Reference Pixels Per Unit: 100

مقدار Match

مقدار پیشنهادی عمومی: 0.5

اگر رابط کاربری شما کاملاً موبایلی و عمودی است: 0.7 تا 1

اگر رابط شما بیشتر بر اساس عرض طراحی شده است: 0 تا 0.3

برای شروع، مقدار زیر انتخاب مناسبی است: Match = 0.5

15. تنظیمات Anchor برای عناصر UI

دکمه پایین صفحه

Anchor: Bottom Center

Pivot: 0.5, 0

لوگو یا عنوان بالای صفحه

Anchor: Top Center

Pivot: 0.5, 1

دکمه بازگشت

Anchor: Top Left

Pivot: 0, 1

پنل تمام صفحه

Anchor Min: 0, 0

Anchor Max: 1, 1

Left: 0

Right: 0

Top: 0

Bottom: 0

از قراردادن عناصر UI با مختصات ثابت بدون Anchor مناسب خودداری کنید.

۱۱. تنظیمات WebGL پروژه

مسیر:

Edit

> Project Settings

> Player

> WebGL

Resolution and Presentation

پیشنهاد:

Default Canvas Width: 1080

Default Canvas Height: 1920

Run In Background: فعال

WebGL Template: EitaaTemplate

Other Settings

برای پروژه سبک و سازگار:

Color Space: Gamma

Auto Graphics API: فعال

Graphics API: WebGL 2

Enable Exceptions: None

اگر پروژه شما وابستگی خاصی به WebGL 1 دارد، باید سازگاری آن را جداگانه بررسی کنید.

Publishing Settings

برای تست اولیه:

Compression Format: Disabled

Decompression Fallback: غیرفعال

مزیت این حالت:

- خطای کمتر هنگام تست؛
 - عدم نیاز به تنظیم Header سرور؛
 - تشخیص ساده‌تر مشکلات فایل‌های Build.
- برای نسخه انتشار، در صورت پشتیبانی هاست:

Compression Format: Brotli

Decompression Fallback: غیرفعال

در این حالت سرور باید Headerهای صحیح ارسال کند.

اگر امکان تنظیم Header ندارید:

Compression Format: Brotli

Decompression Fallback: فعال

16. مراحل Build گرفتن

1. به مسیر زیر بروید:

File > Build Settings

2. پلتفرم WebGL را انتخاب کنید؛

3. روی Switch Platform کلیک کنید؛

4. Scene اصلی را با Add Open Scenes اضافه کنید؛

5. در Player Settings قالب EitaaTemplate را انتخاب کنید؛

6. روی Build کلیک کنید؛

7. یک پوشه خالی مانند این انتخاب کنید:

Builds/EitaaWebGL

خروجی را داخل همان پوشه نگه دارید.

16. ساختار خروجی WebGL

پس از Build، کل این ساختار باید روی هاست آپلود شود:

EitaaWebGL/

└── index.html

└── Build/

│ └── Game.loader.js

│ └── Game.data

│ └── Game.framework.js

│ └── Game.wasm

└── TemplateData/

└── StreamingAssets/

اگر از فشرده‌سازی استفاده شود، ممکن است پسوندها به این شکل باشند:

Game.data.unityweb

Game.framework.js.unityweb

Game.wasm.unityweb

تنها index.html را آپلود نکنید؛ همه پوشه‌ها و فایل‌ها لازم هستند.

17. تست قبل از ثبت در ایتا

ابتدا آدرس را در مرورگر معمولی تست کنید:

<https://example.com/eitaa-app/>

موارد زیر باید درست باشند:

- صفحه بدون خطای SSL باز شود؛

- Unity Loader اجرا شود؛

- فایل‌های پوشه Build پیدا شوند؛
 - بازی یا برنامه نمایش داده شود؛
 - خطای 404 وجود نداشته باشد؛
 - خطای Failed to parse build files وجود نداشته باشد.
- سپس همان URL را داخل ایتا تست کنید.

طبق مستندات ایتا، اجرای برنامه در حال حاضر عمدتاً در نسخه‌های زیر پشتیبانی می‌شود:

- Android
- Desktop Windows

18. ثبت برنامه در ایتا

مراحل کلی:

1. ورود به پنل ایتایار(یا برنامه آن)؛
 2. انجام احراز هویت توسعه‌دهنده؛
 3. ورود به بخش برنامه‌ها؛
 4. ایجاد برنامه اصلی؛
 5. تکمیل اطلاعات برنامه؛
 6. افزودن برنامه؛
 7. ثبت عنوان برنامه؛
 8. ثبت پیوند مستقیم برنامه؛
 9. ثبت آدرس HTTPS وب‌اپلیکیشن؛
 10. فعال‌سازی و تست.
- آدرس مقصد باید نمونه‌ای مانند زیر باشد:

<https://example.com/eitaa-app/>

این URL:

- باید HTTPS باشد؛
- نباید به آدرس دیگری Redirect شود؛
- باید مستقیماً فایل WebGL را اجرا کند؛
- باید از بیرون سرور قابل دسترسی باشد.

19. خطاهای رایج

خطای 404

علت‌های احتمالی:

- پوشه Build ناقص آپلود شده؛
- مسیر فایل‌ها در index.html اشتباه است؛
- نام فایل‌ها تغییر کرده است؛
- حروف بزرگ و کوچک در Linux رعایت نشده است.

خطای Failed to parse build files

علت‌های احتمالی:

- فشرده‌سازی Unity با تنظیمات هاست سازگار نیست؛
 - Headerهای فایل‌های unityweb. ناقص آپلود Decompression Fallback خاموش است؛
 - فایل‌های unityweb. ناقص آپلود شده‌اند.
- برای تست اولیه، این تنظیم را امتحان کنید:

Compression Format: Disabled

SDK ای‌تا در مرورگر عادی کار نمی‌کند

این موضوع طبیعی است. توابع SDK فقط در WebView ای‌تا فعال هستند.

در مرورگر معمولی ممکن است:

علت‌های رایج:

-فایل `jslib` در مسیر صحیح نیست؛

-نام تابع C# با نام JavaScript یکسان نیست؛

-تابع بدون شرط صحیح در Editor فراخوانی شده است؛

- Build قبلی Cache شده است.

مسیر پیشنهادی:

Assets/Plugins/WebGL/EitaaBridge.jslib

UI در موبایل بریده می‌شود

بررسی کنید:

UI Scale Mode = Scale With Screen Size

Reference Resolution = 1080 x 1920

Screen Match Mode = Match Width Or Height

Match = 0.5

همچنین Anchor عناصر را بررسی کنید.

20. نکته بسیار مهم درباره نام API ایتا

فایل Bridge باید دقیقاً با نام شیء JavaScript واقعی SDK ایتا هماهنگ باشد. کد قبلی بر مبنای این ساختار نوشته شده بود:

window.Eitaa.WebApp

اما قبل از انتشار نهایی، این نام باید با نمونه رسمی SDK ایتا تطبیق داده شود. اگر SDK رسمی ایتا شیء را با نام دیگری ارائه کند، مانند:

window.WebApp

یا:

باید تمام فراخوانی‌های Bridge نیز بر همان اساس اصلاح شوند.

بنابراین روند صحیح انتشار این است:

1. ابتدا index.html را روی HTTPS قرار دهید؛

2. آن را در نسخه اندروید ایتا باز کنید؛

3. کنسول یا رفتار ready و expand را بررسی کنید؛

4. سپس Bridge کامل و قابلیت‌های حساس مانند احراز هویت را فعال کنید.

2.1 ابزار Template Configurator

علی‌رغم اینکه می‌توانید فایل index.html را بصورت دستی ویرایش کرده و شخصی سازی قوی ای داشته باشید (برای ورود به برنامه، و صدا زدن شیء)، برای سهولت در شخصی سازی ظاهر صفحه بارگذاری برنامه، یک پنجره ویرایشگر در یونیتی به نام Unita Template Configurator طراحی شده است. این ابزار به شما امکان می‌دهد تا بدون نیاز به ویرایش دستی فایل index.html، پارامترهای زیر را تنظیم کنید و خروجی مورد نظر را دریافت کنید. مسیر دسترسی: Unita > Template Configurator . پارامترهای قابل تنظیم به شرح زیر است:

پارامتر	توضیح
MiniApp Title	عنوان برنامه که در تگ <title> مرورگر نمایش داده میشود.
Loading Text	متنی که در حین بارگذاری به کاربر نشان داده میشود.
Font Name	مقدار font-family در CSS (مثلاً 'Vazir', Tahoma, sans-serif). کاربر میتواند هر فونت وب معتبری را وارد کند.
Text Alignment	تراز متن بارگذاری: Left (چپ)، Right (راست)، Middle (وسط).
Loading text color	رنگ متن بارگذاری (با انتخابگر رنگ).
Background color	رنگ پس زمینه صفحه بارگذاری.
Background image URL	آدرس تصویر پس‌زمینه (اختیاری). در صورت وارد کردن، تصویر به صورت cover و در مرکز قرار میگیرد.
Show loading text	نمایش یا مخفی کردن متن بارگذاری.
Show fullscreen button	نمایش یا مخفی کردن دکمه ورود به حالت تمام صفحه.

22. پارامترهای مهم index.html

22.1. استایل‌های CSS

- استفاده از `display: flex` و `position: absolute` برای قرارگیری صحیح canvas و المانهای بارگذاری.
- تنظیمات `width: 100%` و `height: 100%` برای استفاده از تمام فضای WebView.
- قابلیت تنظیم رنگ پس زمینه، رنگ متن، فونت، تراز متن و تصویر پس زمینه از طریق پارامترهای ورودی.

22.2. متغیرهای یونیتی

- {{{ LOADER_FILENAME }}}
- {{{ DATA_FILENAME }}}
- {{{ FRAMEWORK_FILENAME }}}
- {{{ CODE_FILENAME }}}
- {{{ COMPANY_NAME }}}
- {{{ PRODUCT_NAME }}}
- {{{ PRODUCT_VERSION }}}

این متغیرها در زمان Build با مقادیر واقعی جایگزین میشوند.

22.3. رویدادهای جاوااسکریپت

- بارگذاری اسکریپت loader : پس از بارگذاری کامل، موتور Unity راه اندازی میشود.
- کلیک دکمه تمام صفحه : درخواست حالت تمام صفحه از مرورگر.
- به روزرسانی نوار پیشرفت : با هر تغییر در مقدار پیشرفت، عرض نوار بروزرسانی میشود.

23. نکات دیگر

1. توجه داشته باشید که نمی‌توانید با دبل‌کلیک ساده روی `index.html` آن را در مرورگر باز کنید؛ زیرا مرورگرها به دلیل پروتکل امنیتی CORS اجازه بارگذاری فایل‌های `.wasm` و `.unityweb` را از پروتکل `file:///` نمی‌دهند و با صفحه سفید مواجه می‌شوید. باید یا از روش بارگذاری مستقیم در هاست (Cpanel بهتر است) استفاده کنید، یا افزونه Live Server در VS Code برای بارگذاری مستقیم بر روی هاست.

2. در خروجی Webgl گاهی ممکن است Build سنگین شود. اکثر اوقات، وقتی به مرحله Finished میرسد و روی این مرحله مدت زیادی قفل میشود، بیلد کامل شده و میتوانید Cancel کنید و محتوای کامل (فایل `index.html` و پوشه‌ها و فایل‌های اطراف آن) را مستقیم بر روی هاست/سرور بارگذاری کنید.

3. اگر قصد دارید تا از سنگین بودن فرآیند Build بکاهید:

- حتما پیش از Build گرفتن Clean Build Cache انجام دهید.

- سطح بهینه سازی بالا (Code Optimization) را فعال کنید. به طور پیش‌فرض در حالت Release، کامپایلر سنگین‌ترین بهینه‌سازی‌های WASM را برای کاهش حجم فایل خروجی اعمال می‌کند که زمان بیلد را چند برابر می‌کند. به مسیر `File > Build Settings` بروید. تیک Development Build را فعال کنید. به `Edit > Project > Publishing Settings > WebGL > Player > Settings` بروید. گزینه Optimization را روی Runtime Speed یا Shorter Build Time قرار دهید.

- در مسیر `Player Settings > WebGL > Publishing Settings`: گزینه Compression Format را در زمان توسعه روی Disabled بگذارید. (فقط برای خروجی نهایی انتشار آن را روی Gzip یا Brotli تنظیم کنید).

- آنتی ویروس را موقتا خاموش کنید، یا پوشه پروژه یونیتی و همچنین پوشه UnityHub / Editor را در لیست استثنائات (Exclusions) آنتی‌ویروس و Windows Security قرار دهید.

- سطح پاکسازی کدها (Managed Stripping Level) را تنظیم کنید. تنظیم روی High باعث می‌شود یونیتی کل وابستگی‌های کد را خطبه‌خط تحلیل کند تا متدهای

استفاده‌نشده را حذف کند (Tree Shaking). در مسیر > Player > Player Settings
Other Settings: مقدار Managed Stripping Level را در حین توسعه روی Minimal یا Low بگذارید.

4. نکته مهم درمورد دکمه و حالت تمام صفحه: برخی WebView های موبایل اجازه اجرای کامل Fullscreen API را نمی‌دهند. در این حالت:

- دکمه ممکن است در مرورگر دسکتاپ کار کند.
 - در WebView ای‌تا ممکن است محدود یا بی‌اثر باشد.
 - دستور Eitaa.WebApp.expand() معمولاً گزینه مناسب‌تری برای افزایش فضای نمایشی برنامه است.
- همچنین اجرای ready() و expand() فقط در صورتی انجام می‌شود که گزینه‌های مربوطه در Template Generator فعال باشند.

24. استفاده رایگان از پروژه

همگی مجاز به استفاده از این پروژه هستند. اما لطفاً در انتشار، نام ناشر و توسعه دهنده را فراموش نکنید. بنده رو هم از دعای خیر خودتون بی نصیب نگذارید.

سجاد احمدی نیت

